AI-Powered Learning Management System (LMS) Website

A PROJECT REPORT
for
Major Project-II (CA401P)
Session (2025-26)

Submitted by

Prabhav Kansal
(202410116100143)
Namrta Singh
(202410116100129)
Pallavi Kumari
(202410116100139)

Submitted in partial fulfilment of the
Requirements for the Degree of

MASTER OF COMPUTER APPLICATION

Under the Supervision of
Dr. Amit Kumar
Assistant Professor

Submitted to

Department Of computer Applications
Kiet Group of Institutions, Ghaziabad
Uttar Pradesh -201206

(June-2026)

CERTIFICATE

Certified that **Namrta Singh 202410116100129, Prabhav Kansal 202410116100143, Pallavi Kumari 202410116100139** has/have carried out the project work having **"AI-Powered Learning Management System (LMS) Website"** (**Major Project-CA401P**) for **Master of Computer Application** from Dr. A.P.J. Abdul Kalam Technical University (AKTU) (formerly UPTU), Lucknow under my supervision. The project report embodies original work, and studies are carried out by the student himself/herself and the contents of the project report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Prabhav Kansal (202410116100143)
Namrta Singh (202410116100129)
Pallavi Kumar (202410116100139)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Dr. Amit Kumar
Assistant Professor
Department of Computer Applications
KIET Group of Institutions, Ghaziabad

Dr. Sachin Malhotra
Dean (MCA)
Department of Computer Applications
KIET Group of Institutions, Ghaziabad

ABSTRACT

The project titled **"AI-Powered Learning Management System (LMS) Website"** is a comprehensive full-stack web application designed to transform the traditional learning process into a smart, interactive, and personalized digital experience. Conventional education systems often lack flexibility, accessibility, and personalized learning support, making it difficult for students to learn efficiently. This project addresses these challenges by providing an advanced online platform that enables users to access courses, interact with content, and enhance their learning experience anytime and anywhere.

The system is developed using the **MERN stack**, where **React.js** is used for building a dynamic and responsive user interface, **Node.js** and **Express.js** handle backend operations and API development, and **MongoDB** ensures efficient and scalable data storage. The platform incorporates **AI-based smart search and recommendation features** using modern APIs, allowing users to discover relevant courses quickly based on their interests and queries. Additionally, secure authentication is implemented using **Google OAuth**, and **Razorpay integration** ensures safe and seamless online transactions for premium course enrollment.

The application includes multiple modules such as student dashboard, instructor dashboard, course management system, and admin control panel, providing role-based access and efficient system management. It also supports features like course enrollment, content upload, progress tracking, and real-time updates, making it a complete learning solution.

Keywords: AI-based Learning, Learning Management System, MERN Stack, Smart Search, Online Education, Secure Payment Integration

ACKNOWLEDGEMENTS

Success in life is never achieved alone. I express my deepest gratitude to my thesis supervisor, Dr. Amit Kumar (Associate Professor), for her guidance, assistance, and encouragement throughout my project work. Her insightful ideas, comments, and suggestions have been invaluable in helping me complete this project successfully.

I am also immensely grateful to Dr. Sachin Malhotra, Professor and Head of, the Department of Computer Applications, for his valuable feedback and administrative support on several occasions. I am fortunate to have many supportive friends who have been of great help during critical moments of this journey.

Lastly, I would like to extend my heartfelt thanks to my family members and all those who have provided me with moral support and assistance, both directly and indirectly. Completing this work on time would not have been possible without their unwavering support. Their constant love and encouragement have filled my life with joy and happiness.

Prabhav Kansal
Namrta Singh
Pallavi Kumari

TABLE OF CONTENTS

CHAPTER – 1

INTRODUCTION

1.1 Background

The learning and education sector has experienced significant growth due to increasing demand for online education, digital accessibility, and flexible learning environments. Digital transformation has enabled students to access courses online without being physically present in classrooms. A web-based Learning Management System (LMS) simplifies the entire learning process by allowing users to browse courses, enroll in programs, access study materials, track progress, and manage their profiles—all from a single platform.

The education industry has seen significant transformation in recent years, and with it, the demand for smart and personalized learning solutions has also increased. Online learning platforms have become a vital part of modern education, offering flexibility and accessible alternatives to traditional classroom learning. However, traditional learning systems often suffer from inefficiencies such as lack of personalization, limited accessibility, absence of real-time interaction, and heavy reliance on conventional teaching methods. These limitations not only reduce learning efficiency but also affect user engagement and overall educational outcomes.

1.2 Problem Review

Traditional learning processes involve manual methods, lack of personalized guidance, limited accessibility to resources, and inefficiencies in managing educational content. Students cannot access real-time recommendations or structured learning paths without proper digital support. Administrators and instructors struggle to manage courses, track student progress, and maintain user data manually.

Thus, there is a need for a reliable, automated, user-friendly digital system that handles end-to-end learning processes efficiently.

To address these challenges, the AI-Powered Learning Management System (LMS) has been conceptualized and developed as a full-stack digital solution. This application provides users with a seamless, interactive, and reliable platform to search, browse, and enroll in courses according to their needs. With features such as AI-based course recommendations, real-time course access, secure enrollment, and progress tracking, the application ensures a smooth end-to-end learning process for users.

The project is designed using modern technologies—React.js for building a dynamic and responsive user interface, Node.js with Express.js for handling backend operations, and MongoDB for efficient database management. By adopting these technologies, the system ensures scalability, security, and high performance.

1.3 Objective

The main objectives of this project are:

- To develop a web-based platform for online learning and course management.
- To integrate an AI-based smart search and recommendation system.
- To enable users to view detailed course information and manage enrollments.
- To provide admin tools for managing users, courses, and system activities.
- To build a scalable and secure system using modern technologies.

From an organizational perspective, the system automates repetitive manual tasks, thereby reducing the workload for educational institutions and instructors. Institutions can easily manage course availability, handle enrollments in real-time, and access detailed records of student progress and activities. This leads to improved operational efficiency, reduced errors, and better user engagement.

Beyond solving present-day challenges, the project also envisions future expansions such as AI-driven personalized learning paths, dedicated mobile applications, and advanced analytics to enhance learning outcomes. Thus, this project represents a step toward digital

transformation in education systems, aiming to create a modern, user-centric, and future-ready platform that benefits both learners and educators.

1.4 Key Features

1. User Registration & Login
2. Browse Courses with Filters
3. AI-Based Smart Search and Recommendations
4. Instant Course Enrollment Confirmation
5. Admin Dashboard for Course and User Management
6. Enrollment Cancellation / Course Management
7. Cloud Database (MongoDB Atlas)
8. Responsive UI using React.js

1.5 Scope

The project is designed for educational, commercial, and enterprise use. It supports:

- Online learning platforms and EdTech companies
- Educational institutions and training centers
- Individual instructors and course creators
- College project demonstration
- Scalable future expansion such as multi-course and multi-user support

1.6 Hardware & Software Requirements

Hardware

- Laptop/PC

- Minimum 4GB RAM
- Stable Internet Connection

Software

- VS Code
- Node.js
- Express.js
- React.js
- MongoDB Atlas
- Chrome / Firefox Browser
- GitHub (for version control)

1.7 Background

The rapid growth of online learning platforms inspired the development of this application. This project follows Agile methodology and uses modern web frameworks to ensure fast performance, high reliability, and easy maintainability.

CHAPTER – 2

FEASIBILITY STUDY

A Feasibility Study is conducted to evaluate whether a project is practical, viable, and cost-effective before full-scale development. It analyzes the various factors affecting the success of the proposed system, including economic, technical, operational, and behavioral aspects.

For the AI-Powered Learning Management System (LMS), the feasibility study confirms that the system can be successfully developed using the chosen technologies and can operate efficiently in a real-world educational environment.

2.1 Economic Feasibility

Economic Feasibility examines whether the project is financially viable and whether the benefits outweigh the associated costs.

In this project:

- No significant financial investment is required for development since all major technologies used (React.js, Node.js, Express.js, MongoDB Atlas) are open-source.
- Development does not require commercial tools or expensive software licenses.
- Hosting can be done on low-cost or free-tier platforms such as Netlify (Frontend) and Render / Cyclic / Railway (Backend).
- MongoDB Atlas offers a free shared cluster, which is sufficient for academic and small-scale deployment.
- Hardware costs are minimal, requiring only a basic laptop or desktop with internet access.

- The system reduces manual administrative efforts by automating course management, enrollment, and user tracking, thereby lowering operational costs real-world educational scenarios.

2.2 Technical Feasibility

Technical Feasibility determines whether the required technologies, development platforms, and tools are available and adequate for the successful implementation of the system.

For this project:

- React.js provides a powerful and flexible frontend framework suitable for building dynamic, responsive user interfaces for the LMS.
- Node.js and Express.js offer high performance, non-blocking backend capability ideal for handling multiple user requests such as course access, enrollment, and authentication efficiently.
- MongoDB Atlas provides a reliable, scalable, cloud-based NoSQL database solution for storing user data, course details, and learning records.
- All technologies used are modern, stable, well-documented, and supported by large developer communities.
- The system architecture is highly scalable and can accommodate future enhancements like AI-based recommendations, advanced analytics, real-time collaboration, and mobile application support.

f. No specialized hardware is required; a regular development system with VS Code Node.js installed is sufficient.

2.3 Operational Feasibility

Operational Feasibility evaluates whether the proposed system can function effectively in a real operational environment and whether users can adapt to it easily.

In this project:

a. The system has a clean and intuitive interface, making it simple for users to browse courses, search content, and enroll in programs.

b. Admin and instructor operations such as adding courses, editing content, and managing users are straightforward and require minimal training.

c. The system significantly reduces manual workload for educational institutions and instructors, improving accuracy and efficiency
d. Real-time data synchronization ensures users always see updated course content, enrollment status, and progress without delays.
e. The system supports responsive design, making it accessible across desktops, tablets, and mobile devices.

2.4 Behavioral Feasibility

Behavioral Feasibility assesses how users will accept and adapt to the new system.

For this project:

a. Users are already familiar with online platforms such as online learning websites, educational apps, and video-based platforms, so they can easily understand the course browsing and enrollment process.
b. Minimal learning is required to navigate the system, thanks to its simple interface, form-based inputs, and clear instructions.
c. The system promotes transparency by allowing users to track course enrollments, progress, and learning activities.
d. Admins and instructors also find the system convenient because all operations are available in a centralized dashboard.
e. User feedback can be easily integrated in future versions to improve learning experience and system performance.

CHAPTER – 3

SOFTWARE REQUIREMENTS SPECIFICATION

The Software Requirements Specification (SRS) describes all functional, non-functional, user-based, and system-level requirements necessary for the successful development of the AI-Powered Learning Management System (LMS). The purpose of this chapter is to ensure that every requirement is clearly defined, validated, and understood before development and testing.

SRS acts as a communication bridge between developers, students, instructors, administrators, and stakeholders.

3.1 Functionalities

The core functionalities of the AI-Powered Learning Management System (LMS) are as follows:

1. User Registration & Authentication

a. New users can create an account using name, email, phone, and password.

b. Password is stored securely using hashing/encryption.

c. Existing users can log in to access courses, enrollments, and learning features.

d. Admin and instructors have separate login panels for managing courses, users, and system activities.

2. Course Browsing & Search

a. Users can browse all available courses on the platform.

b. Search functionality based on course name, category, instructor, and price.

c. Filters help narrow down choices using:

- Course Category (Programming, Design, Business, Marketing)
- Price Range
- Course Level (Beginner/Intermediate/Advanced)
- Course Type (Free/Paid)

3. Course Details Viewing

a. Users can click on any course to view:

- Course Price
- Instructor details
- Course level
- Learning modules
- Course duration
- Course description
- Thumbnail/Image gallery

4. Enrollment Management

- Users can enroll in a course by selecting:

i. Course

ii. Enrollment

iii. Payment method

- System automatically checks availability and enrollment status.
- Total amount calculation is displayed for paid courses.
- Enrollment is auto-confirmed once submitted.
- Enrollment details are stored in MongoDB Atlas.

5. My Courses Module

a. Users can view all active and completed courses.

b. Course status displayed (Enrolled/Completed/Cancelled).

c. Users can also manage or cancel enrollment before course access starts.

6. Admin Dashboard

Admin has complete control of the system, including:

I. View total courses, total enrollments, monthly activities.

II. Add new course details.

III. Edit existing course details.

IV. Delete outdated course entries.

V. View all user enrollments.

VI. Approve, update or cancel enrollments.

7. Real-Time Data Updates

I. Whenever a course is enrolled in or cancelled, updated information is fetched instantly.

II. React state ensures UI updates without page refresh.

3.2 User and Characteristics

1. End Users (Students/Learners)

These are individuals who want to enroll in courses for education, skill development, or professional growth.

Characteristics:

a. Basic knowledge of online learning platforms.
b. Internet-enabled device (mobile/laptop).
c. Ability to browse courses and enroll in programs.
d. Expect fast, simple, user-friendly interface.

2. Administrator

The admin manages:

a. Courses
b. Enrollments
c. Users
d. Reports

Characteristics:

a. Must have login credentials.
b. Intermediate computer knowledge.
c. Responsible for keeping data updated.

3. System Requirements for Users

a. A modern browser (Chrome/Firefox).

b. Stable internet connection.

c. Ability to view images and dynamic content.

3.3 Features of Project

This project provides the following major features:

1. Online Car Search

Users can easily search for cars using filters, categories, and sorting options.

2. AI-Based Smart Recommendations

The system provides intelligent course suggestions based on user searches and interests..

3. Secure Login System

a. Password hashing

b. Role-based authentication

c. Error messages for wrong credentials

4. Detailed Course View

Each course has a detailed page containing:

a. Course Specifications

b. Pricing

c. Course thumbnail/videos

d. Terms & conditions

5. Transparent Enrollment System

Users can view:

a. Enrollment date

b. Course details

c. Amount paid

d. Enrollment status

6. Easy Enrollment Cancellation

Users can cancel enrollments instantly, and the cancellation status updates in database.

7. Admin Reporting

Admin gets summary of:

a. Total courses

b. Total enrollments

c. Revenue insights (if extended)

d. Enrollment history

8. Cloud Database Integration

All data is stored in **MongoDB Atlas**, ensuring:

a. Scalability

b. Remote access

c. Data security

3.4 Features of Admin

Admin has full authority and advanced controls such as:

1. Courses Management

a. Add new courses with details such as videos, price, description.

b. Edit course details if any correction or update is needed.

c. Delete outdated or unavailable courses entries.

2. Enrollment Management

a. View all enrollments in system.

b. Approve or cancel bookings.

c. Manually update booking information if required.

3. Dashboard Monitoring

Admin dashboard shows:

a. Total courses on platform

b. Total confirmed enrollments

c. Today's activities

d. Monthly analytics

4. User Management (Optional Extension)

a. View user details

b. Identify active learners

c. Restrict users in case of misuse

3.5 Features of User

Users enjoy the following functionalities:

1. Account Creation & Login

Allows secure login and profile creation.

2. Browse Cars

User can view multiple courses with clear thumbnails and descriptions

3. Car Search Filters

Users can search based on:

a. Price

b. Category

c. Course level

d. Course type

4. Instant Enrollment

Users can enroll in courses easily by selecting courses and confirming details.

5. Enrollment Cancellation

User can cancel enrollment anytime before course access begins

6. My Courses Section

Users can track:

a. Current courses

b. Past courses

c. Canceled enrollment

7. Mobile-Friendly Interface

Responsive UI works smoothly on:

a. Android

b. iPhone

c. Tablets

CHAPTER – 4

HARDWARE & SOFTWARE REQUIREMENTS

Hardware Requirements

1. **Processor**: Intel Core i5 or higher (Quad-Core recommended)
2. **RAM**: Minimum 8 GB (16 GB recommended for smoother development)
3. **Storage**: 500 GB HDD / 256 GB SSD or higher
4. **Display**: 15-inch monitor with minimum resolution of 1366 × 768
5. **Network**: Stable internet connection for hosting and deployment

Software Requirements

1. **Operating System**: Windows / Linux / MacOS
2. **Frontend Technology**: React.js for building dynamic and responsive user interfaces
3. **Backend Technology**: Node.js with Express.js for server-side development
4. **Database**: MongoDB for flexible, NoSQL-based data storage
5. **Web Browser**: Google Chrome / Mozilla Firefox (latest version)
6. **Code Editor / IDE**: Visual Studio Code (VS Code)
7. **Version Control**: Git and GitHub for collaboration and source code management

CHAPTER – 5

PROJECT FLOW

The development of the **AI-Powered Learning Management System (LMS)** follows a structured **Software Development Life Cycle (SDLC)** approach. The key phases are:

1. **Analysis & Planning**
    - Identify the problem statement and limitations of existing car rental systems.
    - Gather requirements from both end-users (students/instructors) and educational institutions.
    - Define the project scope, features, and technology stack.
    - Prepare a development plan with timelines and responsibilities.
2. **Frontend Development**
    - Implement the designed UI using **React.js**.
    - Build reusable components for login, user browsing, enrollment, and payment.
    - Ensure responsiveness across multiple devices (desktop, tablet, mobile).
3. **Backend Development**
    - Set up server-side operations using **Node.js with Express.js**.

- Create RESTful APIs to handle booking requests, user authentication, AI-based search and payments.
- Implement error handling and ensure data security.

4. **Database Integration**
    a. Design and integrate **MongoDB** collections for storing user details, car availability, and booking history.
    b. Ensure proper indexing and relationships for faster data retrieval.
    c. Enable real-time updates for course enrollment and availability status.
5. **Testing & Debugging**
    a. Conduct unit testing for individual modules and integration testing for workflows.
    b. Perform functional, usability, and security testing.
    c. Debug errors and refine the system for stability and reliability.
6. **Maintenance & Updates**
    a. Monitor system performance post-deployment.
    b. Fix bugs, optimize performance, and roll out updates.
    c. Add future enhancements like **AI-based recommendations, mobile application support, and advance analytics** as per roadmap.

CHAPTER – 6

SYSTEM REQUIREMENT

System requirements define the essential conditions, functionalities, constraints, and quality standards needed for the successful development and execution of the AI-Powered Learning Management System (LMS). This chapter outlines both **functional** and **non-functional** requirements and the design goals that guide the structural and technical planning of the system.

6.1 Functional Requirements

Functional requirements describe **what the system should do**. These are directly linked to the core operations of the AI-Powered Learning Management System (LMS).

1. User Registration & Authentication

1. The system must allow new users to create an account.
2. The user must provide valid email, phone number, name, and secure password.
3. Login should validate registered credentials and allow access to the dashboard.
4. Passwords must be stored in a hashed format.

2. User Login & Dashboard

1. Users must be able to log in securely.
2. After login, users should see their dashboard displaying enrolled and completed courses.

3. Browse & Search Courses

1. Users must be able to log in securely.
2. Searching should allow filters based on:
    - Category (Programming, Design, Business, etc.)
    - Price range

- Course Level (if added)

3. Courses must display essential details like course price, instructor, duration, and level.

4. Course Details Page

- Clicking a course must open its full details:
  - Description
  - Features
  - Price
  - Course thumbnail/videos
  - Course modules and content

5. Real-Time Course Access & Availability

a. The system must ensure a user can access enrolled courses instantly.
b. Duplicate or invalid enrollments must be restricted automatically.
c. Course enrollment status must update instantly after enrollment or cancellation.

6. Enrollment Module

a. Users must select course and enrollment options.
b. System must calculate total cost for paid courses.
c. Booking must be stored in MongoDB Atlas.
d. Booking should automatically get the status **"Confirmed"**.

7. Enrollment Cancellation

- Users must be able to cancel enrollment.
- Database must update enrollment status immediately.
- UI should reflect updated status on refresh.

8. Admin Module

Admin must be able to:

a. **Add Courses**

   Upload details such as course title, instructor, price, category, and content.

b. **Edit Courses**

   Update any car information.

c. **Delete Courses**

   Remove outdated or unavailable courses.

d. **Manage Bookings**

   View all enrollments, cancel or verify user enrollments.

e. **Admin Dashboard**

   Summary of:

   - Total Courses
   - Total Enrollments
   - Total Revenue
   - Active Users

9. Report Generation

- Admin should be able to view summarized data:
  - Most enrolled courses
  - Monthly enrollment count
  - Revenue statistics

10. Database Interaction (MongoDB Atlas)

- All information must be stored securely in collections:
  - Users
  - Courses
  - Enrollments
- CRUD operations should be efficient and validated.

6.2 Non-Functional Requirements

These define **how the system should behave**, focusing on performance, security, and user experience.

1. Performance Requirements

- System should respond to user actions within 1–3 seconds.
- Searching, filtering, and enrollment operations must load without lag.
- Backend APIs must be optimized to handle multiple users.

2. Security Requirements

- Passwords must be hashed (e.g., bcrypt).
- Only admin should access admin panel.
- All input fields must be validated.

- Secure communication between frontend and backend.

3. Usability Requirements

- Clean UI with simple navigation.
- Minimal steps for course enrollment.
- Buttons, forms, and pages must be clearly labeled.

4. Reliability Requirements

a. System should work consistently without failure.
b. Enrollment system must always ensure correct enrollment validation.
c. Data retrieved from MongoDB must always be accurate.

5. Availability Requirements

a. System should remain accessible 24/7.
b. Cloud database ensures high uptime availability.

6. Scalability Requirements

a. The system should support:
    - More users
    - More courses
    - More enrollment records
b. MongoDB Atlas supports easy scaling.

7. Portability Requirements

a. Should work on all modern browsers:
    a. Chrome
    b. Firefox
    c. Edge
    d. Brave
b. UI should adapt to desktops, laptops, tablets, and mobiles.

8. Maintainability Requirements

a. Code must be modular with separate components for UI.
b. APIs must be documented for future changes.
c. Database schemas must allow new fields without major changes.

9. Compatibility Requirements

- Frontend should be compatible across devices.

- Backend API must be compatible with any frontend framework if expanded in future.

10. Responsiveness Requirements

a. UI must adjust based on screen width.
b. Pages must remain readable even on mobile devices.
c. Navigation menus must change layout for smaller screens.

6.3 Design Goals

1. Performance Design Goal

a. Ensure fast data loading from MongoDB.
b. Optimize API endpoints.
c. Efficient state management using React hooks and Redux.

2. Security Design Goal

a. Enforce strong authentication.
b. Secure admin-only routes.
c. Validate all inputs on frontend and backend.

3. UI/UX Design Goal

a. Provide visually simple, modern, and responsive interface.
b. Ensure user journeys require minimum clicks.
c. Avoid clutter and maintain consistent theme.

4. Maintainability Design Goal

a. Modular React components.
b. Clean Express route structure.
c. Reusable backend controllers.
d. Centralized error handling.

CHAPTER – 7

SYSTEM DESIGN

System Design is a crucial phase that transforms the gathered requirements into a structured plan for building the AI-Powered Learning Management System (LMS). It describes how the system's components are arranged, how modules interact, how data flows between layers, and how user behavior is supported through meaningful interface design.

Since this project follows an Agile (Iterative) Development Model, the system is designed in incremental stages where improvements, refinements, and functional additions were made continuously after each sprint.

The system design ensures that:

a. The software is modular and easy to maintain.
b. Components are reusable and independently testable.
c. The system supports scalability for future enhancements.
d. UI/UX remains simple, intuitive, and responsive.

7.1 Primary Design Phase

The **Primary Design Phase** focuses on defining the high-level architecture, outlining system modules, identifying user roles, and establishing the main development blueprint.

7.1.1 Identification of System Users

Two primary users were identified:

- **Students/User** – Can browse courses, enroll in programs, and view/manage enrolled courses.
- **Admin** – Manages users, courses, enrollments, and system data.

7.1.2 Identification of System Modules

During this stage, three core modules were finalized:

a. **User Module**
b. **Admin Module**
c. **Course & Enrollment Module**

Each module contains sub-functionalities and API endpoints.

7.1.3 High-Level Architectural Planning

Initial planning involved understanding:

a. Structure of frontend components (React)
b. Routing and navigation flow
c. Backend REST API design (Node + Express)
d. MongoDB collections required: Users, Courses, Enrollments

A general hierarchy was established:

a. UI Layer → API Layer → Business Logic → Database Layer

7.1.4 Database Schema Planning

Early schema planning ensured smooth development:

a. Users collection stores login info, encrypted passwords, user roles, etc.
b. Courses collection stores course details, category, price, instructor, and content.
c. Enrollments collection handles user enrollments, payment details, and course status..

7.1.5 Wireframe & Screen Flow Planning

Before coding, a rough interface blueprint was created:

a. Landing Page
b. Login & Signup
c. Course Listing → Course Details → Enrollment
d. User Dashboard
e. Admin Dashboard

This helped predict user movement inside the system.

7.2 Secondary Design Phase

The Secondary Design Phase focuses on converting high-level ideas into detailed technical design. This includes database structure, API flow, module interaction, and validation logic.

7.2.1 Detailed REST API Planning

APIs were defined for:

a. User Authentication (Register, Login)
b. Course Management (Add, Edit, Delete, View)
c. Enrollment Management (Enroll, Cancel, View Enrollments)
d. Search & Filtering Operations (Category, Price Range, Course Level)

Each API follows:

e. Input validation
f. Error handling
g. JSON-based response structure
h. Role-based access checks

7.2.2 Business Logic Mapping

Detailed logic designed for:

a. **AI-based smart course recommendations**
    - Suggests relevant courses based on user interest
b. **Enrollment management**
    a. Prevent duplicate enrollments
c. **Admin privileges**
    a. Only admin/instructor can modify courses
d. **User actions**
    i. Can only cancel their own enrollments

7.2.3 Component-Based Design (React.js)

Frontend is broken into reusable components:

a. CourseCard
b. EnrollmentForm

c. Sidebar
d. Dashboard Widgets
e. Navbar & Footer
f. Admin Panels

Benefits:

a. Less repeated code
b. Better maintainability
c. Easy to debug and update

7.2.4 Data Flow Design

Flow between client and server:

1. User sends a request from frontend.
2. API receives it and applies validations.
3. Business logic checks rules.
4. MongoDB returns required data.
5. Frontend updates UI accordingly.

7.2.5 Error & Exception Handling Design

a. Invalid login attempts → error message
b. Invalid enrollment requests → blocked
c. Missing form fields → validation messages
d. Deleting enrolled courses → restricted

7.2.6 Security Design

a. Password hashing
b. Protected routes for admin/instructors
c. Form sanitization
d. Token-based authentication

7.3 User Interface Design (UI/UX)

The UI/UX design ensures the system is intuitive, simple, and responsive.

7.3.1 Design Principles Used

a. **Simplicity:** Minimal visual clutter

b. **Consistency:** Uniform button styles, colors
c. **Responsiveness:** Works on mobile, tablet, and desktop
d. **Accessibility:** Clean typography, proper spacing
e. **User-Centered Flow:** Easy navigation for students, instructors and admin

7.3.2 User-Side Screens Designed

a. **Home Page** – Introduction & navigation
b. **Signup Page** – Register new account
c. **Login Page** – Secure authentication
d. **Course Listing Page** – View all available cars
e. **Course Details Page** – Complete car description
f. **Enrollment Page** – Enroll and confirm course access
g. **My Courses Page** – View/manage enrolled courses
h. **User Dashboard** – Personalized learning space

7.3.3 Admin-Side Screens Designed

a. **Admin Login Page**
b. **Admin Dashboard (Statistics Overview)**
c. **Manage Course Page**
d. **Add New Course Page**
e. **Edit Course Page**
f. **Manage Enrollments Page**
g. **View Enrollment Details Page**

7.3.4 UI Technology Stack

a. **React.js** (Component-Based UI)
b. **CSS & Responsive Layout**
c. **React Router** (navigation between pages)
d. **Fetch/AXIOS** for API calls

7.3.5 Overall UI Flow

a. Users → Explore course → View → Enroll → Track progress
b. Admin/Instructor → Login → Manage course → Manage course → Monitor system

CHAPTER – 8

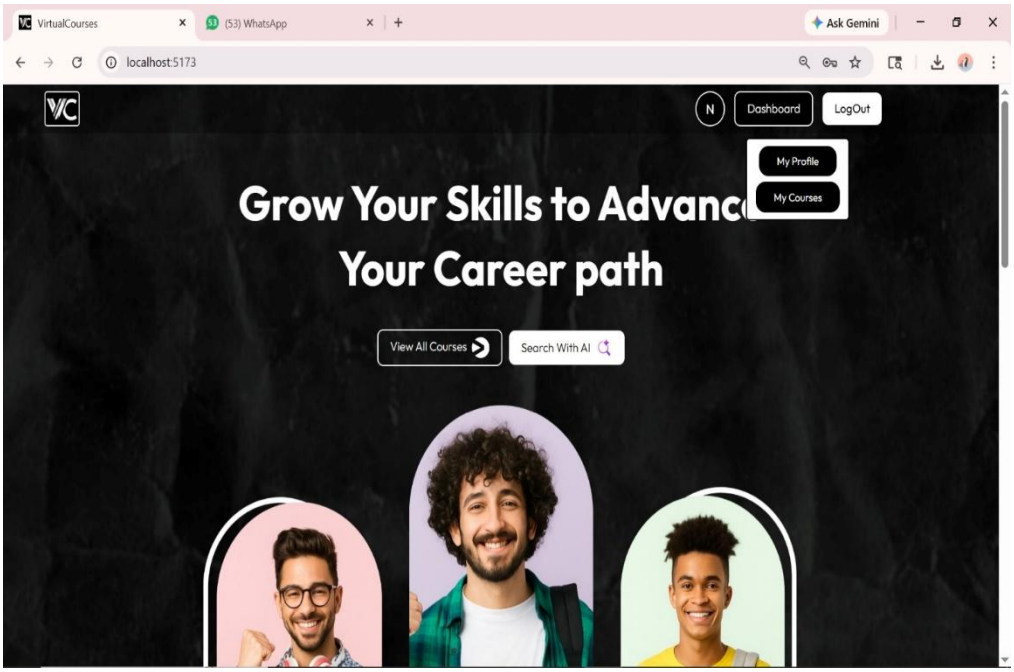
PROJECT OUTCOMES

The successful implementation of the **AI-Powered Learning Management System (LMS)** will lead to the following outcomes:

8.1. User Module → Booking, My Bookings.

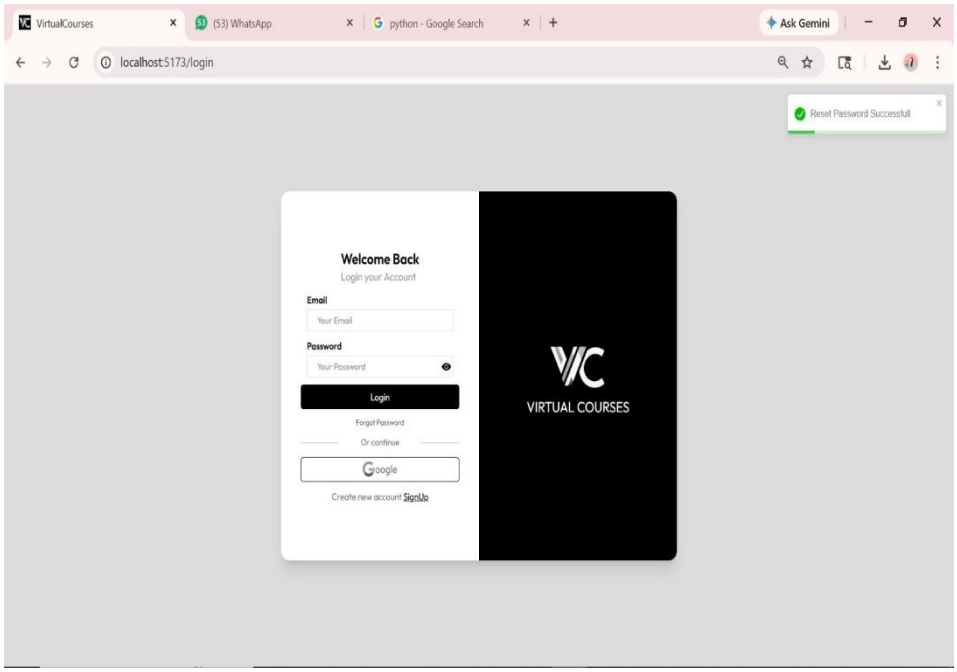
a. **Seamless Vehicle Booking Platform**

Users will be able to browse, search, and book cars online with real-time availability updates, ensuring transparency and convenience (fig. 8.1.1).



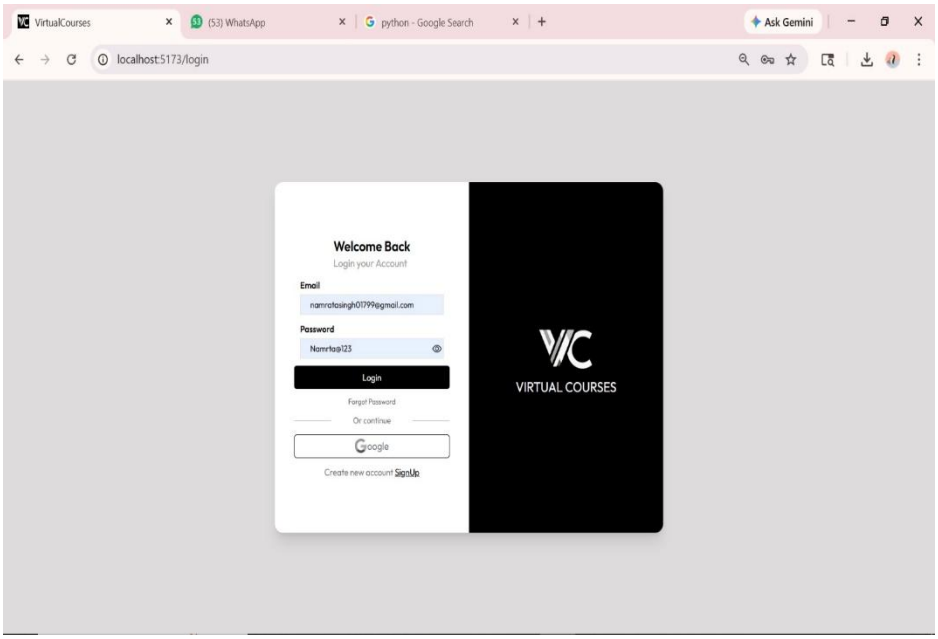
(Fig. 8.1.1)

b. With a responsive **React.js UI**, customers will enjoy an intuitive, interactive, and easy-to-navigate interface across devices (fig.8.1.2) (desktop, tablet, mobile).



(Fig. 8.1.2)

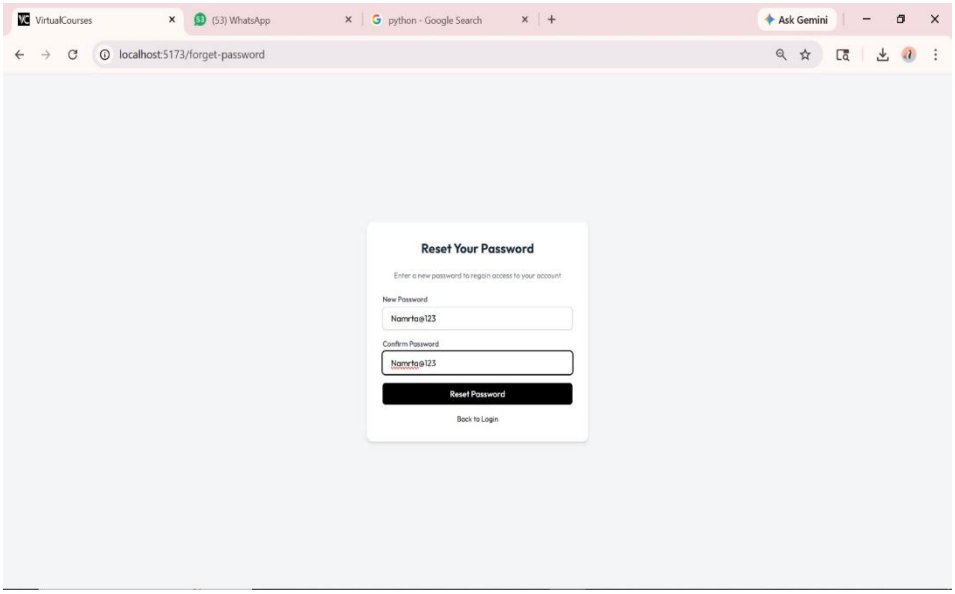
1. The Login Page is the entry point of the Car Rental Booking System for registered users and administrators. This page allows users to securely access the system by providing their registered email address and password. The login form includes input validation to ensure that incorrect or incomplete credentials are not submitted (fig.8.1.3).



(Fig. 8.1.3)

c. **Improved Customer Engagement**

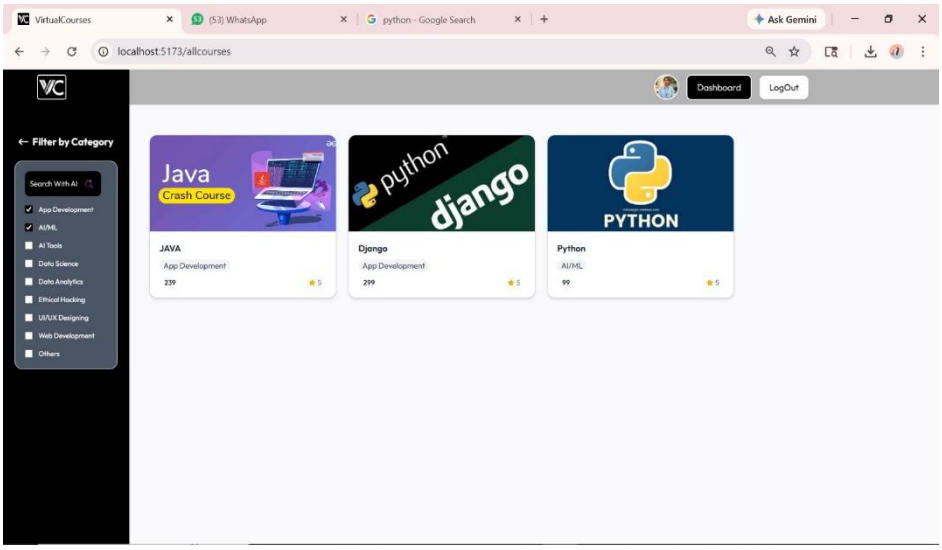
Features like booking history, loyalty program integration (future scope), and personalized suggestions increase user retention and satisfaction (fig.8.1.4).



(Fig. 8.1.4)

d. **Efficient Data Management**

The use of **MongoDB** provides structured storage of booking history, user details, and vehicle availability, enabling quick and accurate data retrieval (fig. 8.1.5).



(Fig. 8.1.5)

e. **Scalability and Flexibility**

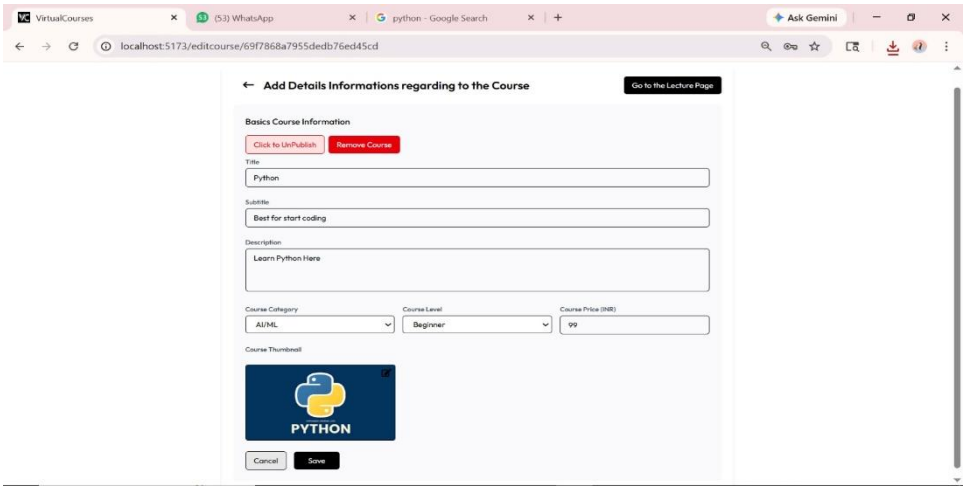
Built on a full-stack architecture, the system is scalable and can be enhanced with future features like AI-driven recommendations, GPS tracking, and mobile apps.

- **Enhanced User Experience**

With a responsive **React.js UI**, customers will enjoy an intuitive, interactive, and easy-to-navigate interface across devices (fig.8.1.6) (desktop, tablet, mobile).

f. **Automation of Agency Operations**

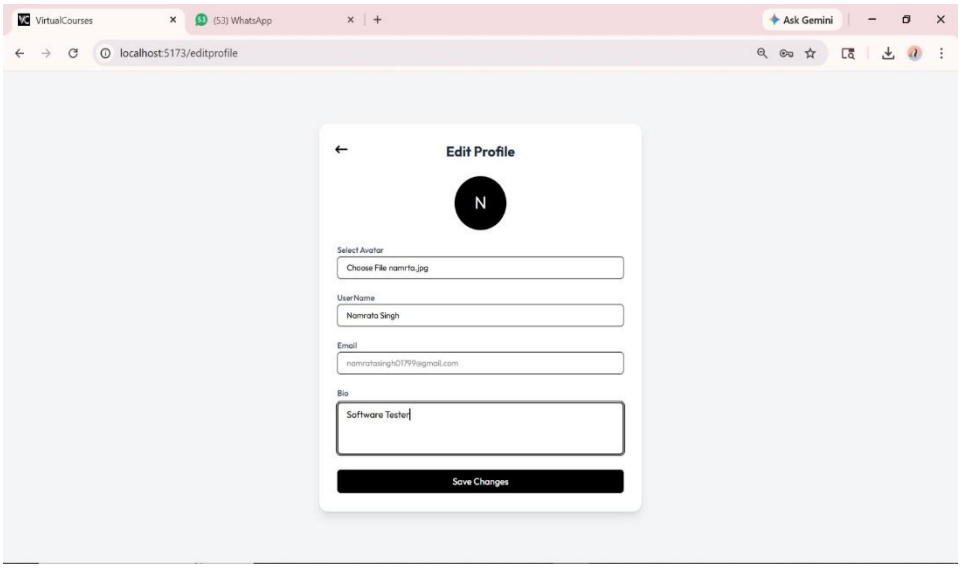
Manual tasks like maintaining records, checking availability, and confirming bookings are automated, reducing workload for rental agencies.



(Fig. 8.1.6)

8.2 Admin Module → Dashboard, Manage Bookings, Car Management.

g. The Admin Module is a core component of the Car Rental Booking System designed to provide complete control and management functionality to the administrator. This module allows the admin to monitor system activities, manage car inventory, and oversee all booking operations efficiently (fig.8.2.1).

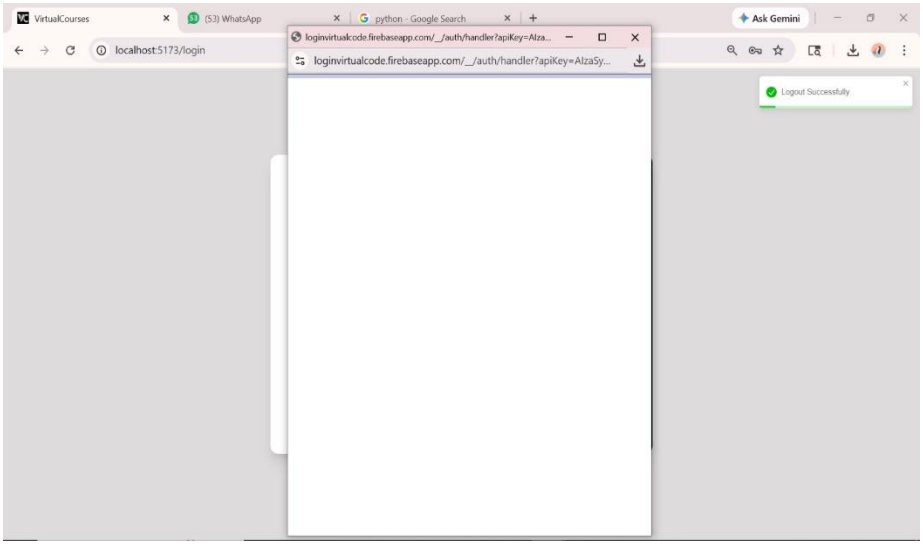


(Fig. 8.2.1)

Contribution to Digital Transformation

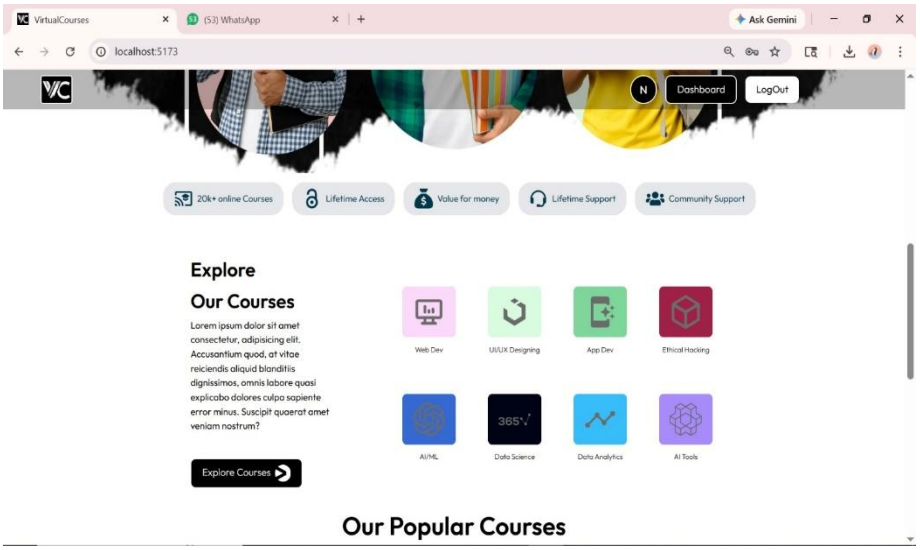
- The project demonstrates how modern web technologies can transform traditional business models, serving as a digital innovation in the transportation sector

1. The Admin Module enables the administrator to add new cars, update existing car details such as price, category, availability, and images, and remove outdated or unavailable vehicles from the system. This ensures that users always have access to accurate and updated car information (fig.8.2.2).



(Fig. 8.2.2)

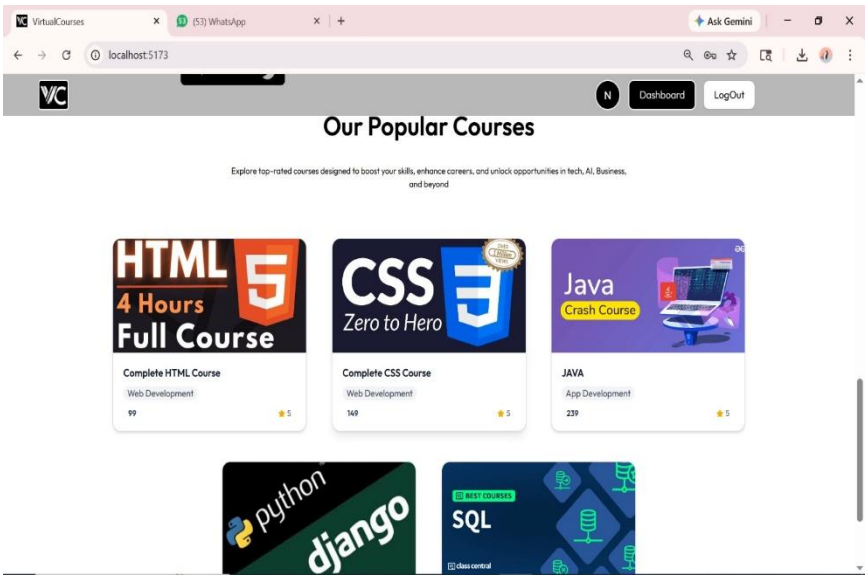
2. This functionality helps maintain proper scheduling, prevents conflicts, and ensures smooth operation of the rental process (fig.8.2.3).



(Fig. 8.2.3)

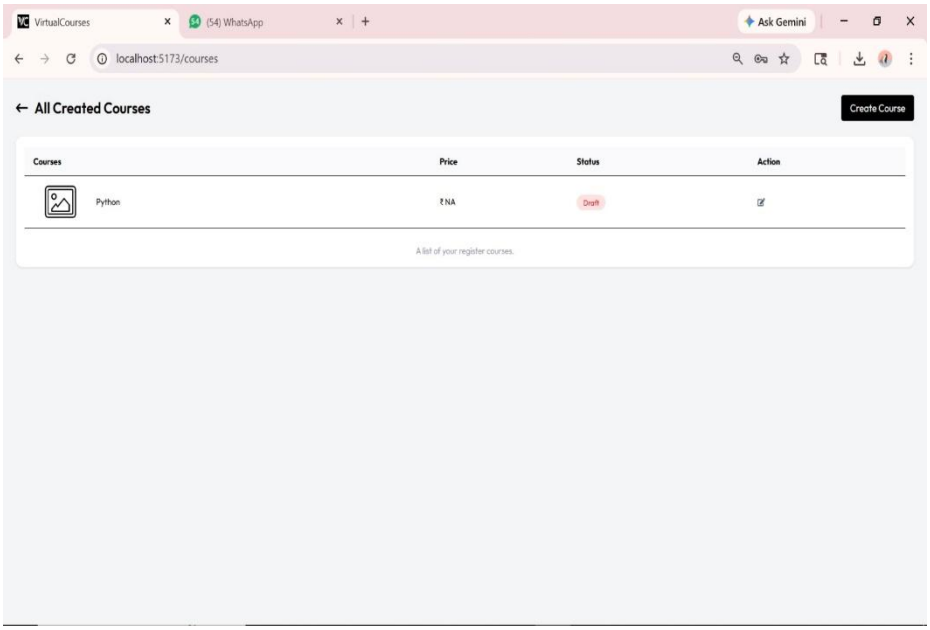
8.3 Database Module

3. The Course **collection** maintains booking records including user ID, car ID, pickup date, return date, booking status, and total rental cost (fig.8.3.1).



(Fig. 8.3.1)

4. The **Course collection** contains car-related data such as car name, model, category, rental price, availability status, and images (fig.8.3.2).



(Fig. 8.3.2)

CHAPTER – 9

TESTING OF AI-POWERED LEARNING MANAGEMENT SYSTEM (LMS)

Software testing is one of the most critical phases in the development life cycle. It ensures that the system performs as expected, delivers accurate results, and functions consistently across all usage scenarios. The AI-Powered Learning Management System (LMS) was tested comprehensively using Manual Testing, with a focus on validating the correctness of each module, identifying functional issues, ensuring smooth user experience, and verifying database integrity.

10.1 Introduction to Testing

Testing is an essential process carried out to verify that the software system meets all the defined requirements and performs reliably in real-world conditions. It ensures the detection of defects, inconsistencies, incorrect logic, performance issues, and failures before the system is deployed.

For the **AI-Powered Learning Management System (LMS)**, testing was conducted with the following objectives:

1. To ensure that user activities such as registration, login, car browsing, filtering, and booking are functioning correctly.
2. To validate admin and instructor operations including adding courses, editing details, deleting courses, and managing enrollments..

3. To confirm that booking rules, especially overlapping date restrictions, operate accurately.
4. To verify that all data is stored correctly in **MongoDB Atlas**.
5. To check responsiveness, usability, and compatibility in multiple browsers.

The primary goal of testing was to deliver a **robust, stable, and user-friendly** application suitable for real-world use.

10.2 Types of Testing Conducted

Multiple testing types were applied to validate all modules of the AI-Powered Learning Management System. Each testing type contributed to improving system reliability.

1. Functional Testing

Ensures every function of the application works exactly as intended according to the requirement specification.

Key areas tested:

- User registration and authentication
- Admin/instructor login and access verification
- Course listing display and filtering
- Course enrollment and enrollment cancellation
- Admin operations (Add/Edit/Delete cars, Approve/Cancel enrollments)
- Viewing and updating database records

2. Usability Testing

Focuses on the ease of use, intuitiveness, and smoothness of the interface.

Tasks observed:

a. Whether a new user can book a car without help
b. Clarity of navigation menus
c. Readability of text, labels, and instructions
d. Responsiveness on different screen sizes

3. Validation Testing

Ensures that the system accepts only valid inputs and rejects invalid ones.

Examples:

a. Preventing duplicate emails during registration

b. Restricting login with incorrect credentials
c. Preventing duplicate course enrollments
d. Ensuring search filters accept valid values

4. Compatibility Testing

Validates how the system performs under different browsers.

Browsers tested:

a. **Google Chrome**
b. **Mozilla Firefox**

The UI was checked for consistency in:

a. layout
b. colors
c. fonts
d. alignment
e. button responsiveness

5. Database Testing

Ensures that all CRUD operations are correctly performed in **MongoDB Atlas**.

Database tests included:

- Insertion of new users
- Updating enrollment status
- Storing course and payment details
- Deleting car records
- Fetching latest information

Database accuracy is essential because course access and enrollments depend on real-time data.

10.3 Test Plan and Execution

The test plan defines the scope, objectives, resources, responsibilities, and activities of the testing process.

Testing Objectives

a. Ensure accuracy in user and admin modules
b. Verify enrollment validity and restrictions

c. Validate API responses and data flow
d. Identify usability and navigation issues

Testing Strategy

a. Manual execution of all test scenarios
b. Use of real-time test data: valid, invalid, edge cases
c. Browser testing on Chrome and Firefox
d. Database validation using MongoDB Atlas dashboard
e. Logs inspection in backend terminal

Test Environment

a. Frontend: React.js
b. Backend: Node.js + Express.js
c. Database: MongoDB Atlas Cloud Cluster
d. OS Used: Windows 10
e. Browsers: Chrome & Firefox

Execution Approach

Testing was carried out in phases:

Phase 1: Module Testing

Each module—User, Admin, Enrollment—was tested independently.

Phase 2: Integration Testing

Combined modules were tested together:

a. Login → Dashboard → Course List → Enrollment

Phase 3: System Testing

Full system flow tested end-to-end.

Phase 4: Regression Testing

Re-tested modules after bug fixes.

This systematic approach ensured reliability and high product quality.

10.4 Results and Reports – Manual Test Cases (15 Test Cases)

Below are the **15 detailed test cases**, covering the entire system.

TABLE 10.4.1: Manual Test Cases

Test Case ID	Test Scenario	Input	Expected Result	Actual Result	Status
TC01	Register new user	Valid user info	User registered	As expected	PASS
TC02	Register with existing email	Duplicate email	Error displayed	As expected	PASS
TC03	Login with correct credentials	Valid email, password	Redirect to dashboard	Works correctly	PASS
TC04	Login with wrong password	Wrong password	Error message shown	Works correctly	PASS
TC05	Admin login	Admin credentials	Access admin dashboard	Works correctly	PASS
TC06	Search course by category	Select Programming	Only related courses shown	Correct output	PASS
TC07	Filter course by price	Range 1000–2000	Course in range displayed	Works correctly	PASS
TC08	View course details	Click course card	Course details displayed	Works correctly	PASS
TC09	Enroll in course	Valid enrollment	Enrollment confirmed	Works correctly	PASS
TC10	Duplicate enrollment attempt	Already enrolled course	vEnrollment blocked	Works correctly	PASS
TC11	Cancel enrollment	Click cancel	Status → Cancelled	Updated correctly	PASS
TC12	Add course (Admin)	Course details	Course added	Works correctly	PASS
TC13	Edit course	Price updated	Course updated	Works correctly	PASS

TC14	Delete course	Delete button	Course removed	Works correctly	PASS
TC15	Validate database entry	Enrollment entry	Saved in MongoDB	Correctly saved	PASS

10.5 Challenges Faced

During the testing phase, several challenges were encountered:

1. Managing Real-Time Enrollment Updates

Ensuring accurate detection of overlapping bookings required complex logic and multiple rounds of validation.

2. State Management Issues in React

Certain UI components were not updating instantly due to asynchronous state updates. This was fixed using improved state handling.

3. Browser Compatibility Adjustments

Certain CSS components displayed differently in Firefox compared to Chrome, requiring UI refinements.

4. Database Sync Delays

MongoDB Atlas sometimes took a few seconds to reflect updated booking data, requiring retesting.

5. API Error Handling

Clear and user-friendly error messages had to be written for all scenarios (invalid login, invalid booking dates, etc.).

CHAPTER – 10

PROPOSED TIME DURATION

The development of the **AI-Powered Learning Management System (LMS)** is planned to be completed within **7–8weeks**, following a systematic timeline. The tentative phase-wise duration is:

1. **Phase 1 – Requirement Analysis & Planning (2 weeks)**

   Understanding the project requirements and educational workflow.

   b. Gathering requirements from students, educators, and administrators.

   c. Finalizing project modules, technology stack, and database structure.

   d. Preparing the project roadmap and development timeline.

2. **Phase 2 – Frontend Development (2 weeks)**

   a. Designing responsive user interfaces using React.js.

   b. Developing modules such as Login, Signup, Dashboard, and Course Pages.

   c. Creating reusable React components for better maintainability.

   d. Ensuring responsive design for desktop, mobile, and tablet devices.

3. **Phase 3 – Backend Development (3 weeks)**

1. Setting up **Node.js with Express.js** server.
2. Creating RESTful APIs for booking, authentication, and payments.
3. Implementing security features like session management and error handling.

4. **Phase 4 – Database Integration (3 weeks)**

2 Designing and integrating **MongoDB** collections.

3 Managing user data, car availability, and booking records.

4 Enabling real-time updates and efficient queries.

5. **Phase 5 – Testing & Debugging (3 weeks)**

- Conducting unit testing, integration testing, and functional testing.
- Fixing bugs and optimizing the application for performance.
- Performing usability and security testing.

6. **Phase 6 – Maintenance & Updates (1 weeks, ongoing)**

- Monitoring system performance.
- Addressing post-deployment issues.
- Rolling out updates and preparing for future enhancements such as **AI recommendations, and loyalty programs**.

CONCLUSION

The **AI LMS – Virtual Courses Platform** developed as part of this major project demonstrates the successful implementation of modern full-stack web technologies including React.js, Node.js, Express.js, and MongoDB Atlas to build an intelligent, responsive, and scalable online learning platform. The primary objective of this project was to simplify and modernize the traditional education system by providing a centralized digital platform where educators can create and manage courses while students can explore and access learning content from anywhere. The project was initiated after identifying major challenges in traditional educational systems such as limited accessibility, manual course management, lack of centralized learning resources, poor scalability, and restricted interaction between educators and learners. Conventional learning systems often fail to provide flexible and personalized learning experiences. The developed AI LMS platform successfully addresses these challenges by digitizing the educational workflow and automating major operations including user registration, authentication, course creation, dashboard management, profile handling, and online course access.

The application follows a structured full-stack architecture where React.js is used for building a dynamic, component-based, and responsive user interface. The frontend design ensures smooth navigation, fast rendering, and compatibility across desktops, laptops, tablets, and mobile devices. The backend, developed using Node.js and Express.js, manages RESTful APIs, authentication, routing, database communication, and server-side operations efficiently. MongoDB Atlas is used as the cloud-hosted NoSQL database to store user information, course details, profile data, and platform content securely.

The system architecture follows modular development principles which improve maintainability, scalability, and code reusability. The project includes multiple modules such as authentication module, dashboard module, course management module, profile management module, and AI learning module. The responsive dashboard allows educators to upload and manage virtual courses while students can browse and explore educational content seamlessly.

The project was tested thoroughly using manual testing methodologies. Major functionalities such as signup, login, role management, profile updates, course creation, course editing, dashboard rendering, responsive design, database operations, and browser compatibility were validated successfully. Testing confirmed that the system performs accurately and remains stable under different usage scenarios. Compatibility testing across modern browsers such as Google Chrome and Mozilla Firefox ensured consistent UI behavior and proper responsiveness.

From an educational and technical perspective, this project provided practical exposure to Full-stack web application development. REST API implementation. Cloud database integration using MongoDB Atlas \nd. React.js component-based architecture.

Responsive web design principles. Authentication and authorization systems. Real-world dashboard and course management implementation. Scalable system architecture planning. The knowledge and experience gained during the development process align with current industry standards and strengthen professional development skills.

REFERENCES

1. React Official Documentation – React Official Documentation
2. Node.js Documentation – Node.js Documentation
3. Express.js Documentation – Express.js Documentation
4. MongoDB Atlas Documentation – MongoDB Atlas Documentation
5. MDN Web Docs – MDN Web Docs
6. W3Schools Tutorials – W3Schools Tutorials
7. FreeCodeCamp Full Stack Tutorials – FreeCodeCamp
8. GeeksforGeeks MERN Stack Notes – GeeksforGeeks
9. GitHub Documentation – GitHub Docs
10. Postman API Testing Documentation – Postman Learning Center
11. Netlify Deployment Documentation – Netlify Docs
12. Vercel Deployment Documentation – Vercel Docs
13. JWT Authentication Guide – JWT.io
14. REST API Best Practices – REST API Tutorial
15. CSS Tricks Responsive Design Guide – CSS Tricks
16. MongoDB University Resources – MongoDB University
17. Tailwind CSS Documentation – Tailwind CSS Docs
18. Redux Toolkit Documentation – Redux Toolkit Docs
19. Firebase Authentication Documentation – Firebase Auth Docs
20. Cloudinary Documentation – Cloudinary Docs
21. Razorpay Payment Gateway Docs – Razorpay Docs

22. Google OAuth Documentation – Google OAuth Docs
23. Axios HTTP Client Documentation – Axios Docs
24. React Router Documentation – React Router Docs
25. NPM Official Documentation – NPM Docs
26. Visual Studio Code Documentation – VS Code Docs
27. HTML5 Official Reference – HTML5 Reference
28. CSS3 Documentation – CSS Reference
29. JavaScript Documentation – JavaScript Docs
30. Bootstrap Documentation – Bootstrap Docs
31. Webpack Documentation – Webpack Docs
32. Babel JavaScript Compiler Docs – Babel Docs
33. Socket.IO Documentation – Socket.IO Docs
34. Mongoose ODM Documentation – Mongoose Docs
35. RESTful Web Services Guide – RESTful API Guide
36. JSON Web Token Security Guide – JWT Security Guide
37. Git Version Control Documentation – Git Documentation
38. Linux Command Documentation – Linux Documentation Project
39. Agile Software Development Notes – Agile Manifesto
40. Scrum Methodology Guide – Scrum Guide
41. UI/UX Design Principles – Interaction Design Foundation
42. Responsive Web Design Course – Responsive Web Design
43. MERN Stack Tutorial – MERN Stack Tutorial
44. Coursera E-Learning Research Articles – Coursera Research
45. Udemy Online Learning Platform – Udemy
46. edX Digital Learning Resources – edX
47. Khan Academy Learning Resources – Khan Academy
48. IEEE Research Papers on E-Learning – IEEE Xplore
49. Springer Research Articles – SpringerLink
50. ResearchGate Academic Papers – ResearchGate